Assignment for ASI : Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles Lakshminarayanan et al. (2017)

Chaoui Kouraichi Ahmed Taha

Abstract

This report summarizes and reproduces the paper Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles by Lakshminarayanan et al. (2017). The paper introduces a non-Bayesian approach to estimate uncertainty in neural networks. It relies on three main ideas: (1) training networks to predict both mean and variance using a proper scoring rule, (2) applying adversarial training to smooth predictions, and (3) combining independently trained networks into an ensemble. In this report, we summarize these concepts, reproduce the 1D toy regression experiment, and discuss the strengths and limitations of the method. The code is available at: `https://github.com/chaoui-ahmed/ASI-PAPER/`.

1 Introduction

Reliable uncertainty estimation is essential when deploying machine learning models in critical domains like medical diagnosis or autonomous driving. An effective model should be confident about familiar inputs, but it must also express high uncertainty when dealing with unfamiliar or out-of-distribution (OOD) data.

Typically, uncertainty is modeled using Bayesian deep learning, where we place a prior on the weights θ to compute the posterior $p(\theta \mid \mathcal{D})$. However, exact Bayesian math is too hard to compute for deep neural networks. Because of this, researchers use approximations like Variational Inference (VI), MCMC, the Laplace approximation, or MC-Dropout.

As an alternative, Lakshminarayanan et al. propose deep ensembles. This method is simpler, scales easily to large datasets, and requires minimal changes to standard neural network training, while still providing strong uncertainty estimates.

2 Summary of the Paper

2.1 Problem Setup

Given a training dataset $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, a standard neural network typically outputs a single point estimate. Here, the goal is to predict a full *predictive distribution* $p_\theta(y \mid \mathbf{x})$.

For regression tasks, the network predicts both a mean $\mu_\theta(\mathbf{x})$ and a variance $\sigma_\theta^2(\mathbf{x}) > 0$. Together, they form a Gaussian distribution where the variance depends on the input:

$$p_\theta(y \mid \mathbf{x}) = \mathcal{N}\big(y \;\big|\; \mu_\theta(\mathbf{x}),\ \sigma_\theta^2(\mathbf{x})\big)\,. \tag{1}$$

For classification tasks with K classes, the network outputs a categorical distribution using a standard softmax layer.

Preprint.

2.2 Proper Scoring Rules

A proper scoring rule $S(p_\theta, (y, \mathbf{x}))$ measures the quality of a predictive distribution. It gives the best score only when the predicted distribution exactly matches the true data distribution ($p_\theta = q$). This encourages the network to be well-calibrated.

The negative log-likelihood (NLL) is a well-known proper scoring rule. For a Gaussian output, the training loss becomes:

$$\mathcal{L}(\theta) = \sum_{n=1}^{N} \left[\frac{\log \sigma_\theta^2(\mathbf{x}_n)}{2} + \frac{(y_n - \mu_\theta(\mathbf{x}_n))^2}{2\sigma_\theta^2(\mathbf{x}_n)} \right]. \tag{2}$$

2.3 Adversarial Training for Smoothing

The authors propose adding small intentional noises to the inputs during training to make the predictions smoother:

$$\mathbf{x}'_n = \mathbf{x}_n + \epsilon \, \text{sign}(\nabla_{\mathbf{x}_n} \mathcal{L}(\theta, \mathbf{x}_n, y_n)), \tag{3}$$

where ϵ is a small value (like 1% of the input range). Training with the combined loss $\mathcal{L}(\theta, \mathbf{x}_n, y_n) + \mathcal{L}(\theta, \mathbf{x}'_n, y_n)$ forces the network to maintain stable predictions even when the input changes slightly.

2.4 Ensemble Combination

We train M different networks $\{\theta_m\}_{m=1}^M$. Each network is trained on the same dataset, but starts with different random weights. The final prediction is calculated by averaging their outputs:

$$p(y \mid \mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} p_{\theta_m}(y \mid \mathbf{x}). \tag{4}$$

For regression, averaging M Gaussians results in a mixture of Gaussians. We approximate this with a single Gaussian with the following mean and variance:

$$\mu_*(\mathbf{x}) = \frac{1}{M} \sum_m \mu_{\theta_m}(\mathbf{x}), \qquad \sigma_*^2(\mathbf{x}) = \frac{1}{M} \sum_m \left[\sigma_{\theta_m}^2(\mathbf{x}) + \mu_{\theta_m}^2(\mathbf{x}) \right] - \mu_*^2(\mathbf{x}). \tag{5}$$

The total predictive variance decomposes into two parts:

- **Aleatoric uncertainty:** $\frac{1}{M} \sum_m \sigma_{\theta_m}^2(\mathbf{x})$ this represents the natural noise in the data, which cannot be reduced by collecting more data.
- **Epistemic uncertainty:** $\frac{1}{M} \sum_m \mu_{\theta_m}^2(\mathbf{x}) - \mu_*^2(\mathbf{x})$ this represents the disagreement between the different models, showing the model's lack of knowledge in regions without data.

2.5 Why Ensembles Work: Connection to Bayesian Inference

Because of how neural networks work, different random starting weights lead them to find different solutions. While approximate Bayesian methods (like VI) tend to focus on a single solution, deep ensembles naturally capture many different ones.

For instance, MC-Dropout simulates uncertainty by randomly turning off neurons. However, because all dropped versions share the same core weights, the explored solutions are quite similar. In contrast, deep ensembles explore completely different weight paths, making them much better at detecting unknown data.

Also, math theory shows that very wide neural networks behave like Gaussian processes. This confirms that random initialization naturally creates a good diversity of models.

2.6 Real-World Regression Benchmarks

The authors tested deep ensembles on 10 real-world UCI regression datasets, comparing them against PBP and MC-Dropout.

The results show that deep ensembles consistently match or beat the baselines. They achieved the best test log-likelihood on 6 out of 10 datasets. This improvement is especially visible on larger datasets, where training independent models generates more useful diversity.

2.7 Classification Experiments

For classification tasks, the authors tested on datasets like MNIST and CIFAR-10 using $M = 5$ networks. They noticed that both ensembling and adversarial training independently reduce errors, and combining them gives the best performance.

They also tested out-of-distribution (OOD) detection by training the model on MNIST (digits) and testing it on NotMNIST (letters). Deep ensembles successfully showed high uncertainty (entropy) for the unseen letters, whereas MC-Dropout was often overconfident on incorrect predictions.

2.8 Accuracy as a Function of Confidence

A practical evaluation is measuring accuracy based on the confidence threshold. If we filter out uncertain predictions and only keep the confident ones, the accuracy of deep ensembles goes up very quickly. This proves that their confidence scores are reliable.

In contrast, MC-Dropout assigns high confidence to many incorrect predictions. This makes it unreliable for safety-critical applications where a model should abstain rather than make confident mistakes.

3 Reproduced Results

3.1 Toy Regression Experiment

We reproduced the 1D toy regression experiment from Section 3.2 of the paper. The dataset contains 20 training points generated using $y = x^3 + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 9)$.

Implementation details. The experiment was implemented in Python using the JAX framework. Each network in the ensemble is an MLP with a single hidden layer of 50 units and ReLU activations. The output layer predicts the mean $\mu_\theta(\mathbf{x})$ and the log-variance $\log \sigma^2_\theta(\mathbf{x})$. We minimized the NLL loss using the Adam optimizer (learning rate 0.01) for 5000 epochs.

Experimental protocol

1. **Single NN with MSE:** We train a single network to minimize MSE. It provides a point prediction, without any variance.
2. **Single NN with NLL:** We train a single network to minimize the NLL loss. It learns to predict variance.
3. **Ensemble of $M = 5$ NNs with NLL:** We train 5 networks independently with different random seeds, then combine them.

Results.
The reproduction confirms the paper's findings:

- A single network trained with MSE only provides a point prediction, without any uncertainty estimate.
- A single network trained with NLL learns to predict variance, but it remains overconfident in regions far from the training data.
- The ensemble of 5 networks performs well. The predicted uncertainty is small near the training data and grows a lot in unexplored areas, meaning the model properly recognizes its own ignorance.

Figure 1 shows the three experiments side by side. The growing uncertainty band in the ensemble plot (right) is the key result.

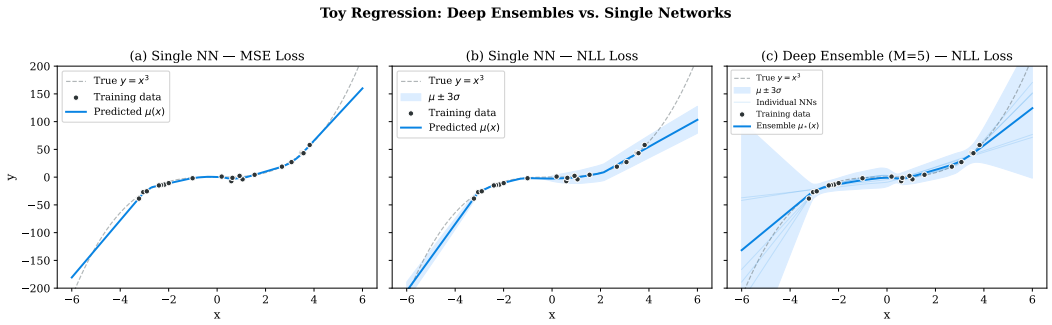
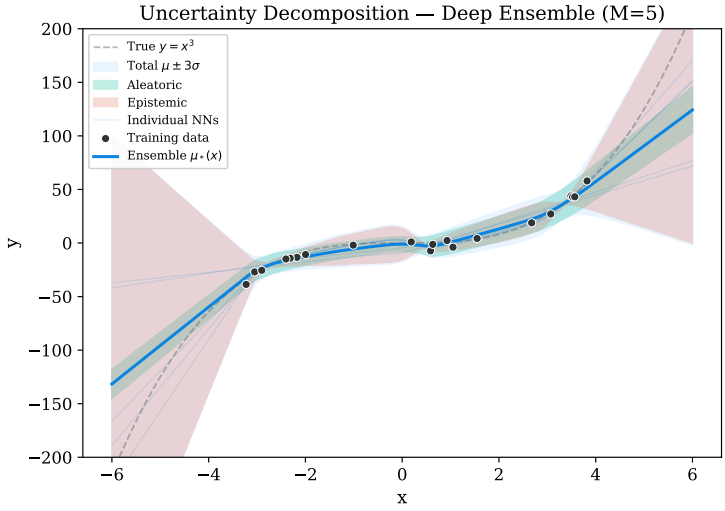


Figure 1: Toy regression experiment ($y = x^3 + \epsilon$, 20 training points). **(a)** Single NN with MSE provides only a point prediction. **(b)** Single NN with NLL learns variance but remains overconfident far from the data. **(c)** Deep ensemble ($M = 5$) with NLL produces well-calibrated uncertainty that grows away from training data. Shaded regions show $\mu \pm 3\sigma$.

Figure 3.1 shows the uncertainty decomposition. The aleatoric part (green) stays flat because it represents the constant data noise $\epsilon \sim \mathcal{N}(0, 9)$. The epistemic part (red) grows outside the training range, reflecting how much the 5 models disagree in unknown areas.



The main advantage of deep ensembles is getting this robust uncertainty estimation using standard neural networks, without requiring complex Bayesian inference.

3.2 Comparison with Bayesian Approaches

Table 1: Comparison of uncertainty methods studied.

Method	**Scalability**	**Multimodal**	**Simplicity**
Exact Bayes	Low	Yes	Medium
VI / Mean-field	High	No	Medium
MCMC	Low	Yes	Low
Laplace	High	No	Medium
MC-Dropout	High	No	High
Deep Ensembles	**High**	**Yes**	**High**

Deep ensembles are simple to implement, scale well to large datasets, and successfully capture multiple solutions. Other methods like VI or Laplace tend to focus on a single solution. MCMC is often too slow and complex. Deep ensembles are completely parallel. The only downside is that training M networks increases the memory and computing cost by a factor of M.

4 Discussion

4.1 Strengths of the Approach

Simplicity and practicality. The method requires only three simple modifications to standard training: predicting the variance, using the NLL loss, and training M models. It avoids complex probability distributions and convergence checks.

Scalability. Training can be fully run in parallel. The authors successfully applied the method to large datasets like ImageNet, which is generally too heavy to compute for traditional Bayesian methods.

Strong OOD detection. The method performs very well at out-of-distribution detection, consistently expressing high uncertainty on unseen classes.

4.2 Limitations

No strict mathematical prior Unlike formal Bayesian methods, deep ensembles do not use an explicit prior $p(\theta)$. The diversity relies on the random starting weights.

Computational cost Training and storing 5 independent networks increases the computing cost and memory size by a factor of 5. To fix this at test time, the authors propose knowledge distillation, where a single student network is trained to mimic the ensemble's output.

4.3 Conclusion

What worked well: Using JAX for this assignment was a good choice. Jax.numpy works almost exactly like standard NumPy. It also was really helpful because I needed to make sure the 5 models in the ensemble were initialized completely differently. As we can see in the figures, this setup was enough to successfully reproduce the main toy experiment from the paper.

What was challenging: The hardest part of the code was dealing with numerical the NLL loss equation that divides by the variance and takes its log. During training, if the model predicts a variance very close to zero, the loss explodes, we get NaN values, and the training crashes. To fix this, I really had to follow the paper's trick and use a softplus function with a small offset for the variance output. Figuring this out and debugging it took some time.

What could be improved: First, we only reproduced the 1D toy dataset. It would be interesting to run the code on real-world datasets to see how the code scales. Second, I skipped the FGSM because it was a bit complex to implement, but adding it would make the reproduction more complete.

5 Use of AI

- Overleaf AI for LaTeX syntax (lists, fonts, layouts, formulas, etc.).
- Suggestions to refine the phrasing and maintain an appropriate academic tone.
- Advice on selecting the right experimental setup given my resources.
- Assistance with writing and debugging the JAX code.